Project Case Study: GeoFy

Real-Time Geofencing & Field Agent Tracking System

1. Executive Summary

GeoFy is a full-stack location platform designed to automate the logging of field agent visits. By combining Geospatial querying with Real-time GPS tracking, the application eliminates manual check-ins, providing accurate data on when an agent arrives at and departs from a designated property.

2. The Core Logic: How It Works

A. Real-Time Tracking (Frontend)

The client-side uses the Browser's Geolocation API (watchPosition). As the agent moves, their coordinates are captured and sent to the backend every few seconds.

- Persistent Navigation: Leaflet Routing Machine calculates the path between the AgentLocation and PropertyLocation.
- State Management: LocalStorage is used to ensure that if the agent's browser crashes or refreshes, the active trip and map view are restored immediately.

B. Geospatial Validation (Backend)

When coordinates reach the Node.js server, they are passed into a MongoDB query using the $near operator.

- The 50m Rule: The server checks if the agent's current point is within a 50-meter radius of the selected property's 2dsphere index.
- The Logic Gate:
    - **IF Inside + No Active Log**: Create a new "Active" log (Check-in).
    - **IF Inside + Existing Active Log**: Do nothing (Stay checked-in).

- **IF Outside + Existing Active Log**: Mark log as "Completed" and timestamp the exit (Check-out).

3. Data Flow Architecture

Admin Entry: Admin saves a property (GeoJSON Point).

Agent Selection: Agent selects a property from the list.

Live Sync: Frontend pushes coordinates **==>** Backend validates **==>** Database updates logs.

Admin Oversight: Admin views the visitingLogs collection to see entry times and exit times of the visit.

4. Technical Challenges & Solutions

Accurate Logging :

```
// ========================================= AGENT ==================================
app.post('/api/check-location', async (req, res) => {
    const { agentLng, agentLat, agentName, propertyid } = req.body;
    try {
        const property = await db.collection("geofences").findOne({
            _id: new ObjectId(propertyid),
            location: {
                $near: {
                    $geometry: {
                        (property) coordinates: number[]
                        coordinates: [parseFloat(agentLng), parseFloat(agentLat)]
                    },
                    $maxDistance: 50
                }
            }
        });

        if (property) {
            const existingLog = await db.collection('visitingLogs').findOne({
                agentName: agentName,
                propertyId: property._id,
                status: "Active"
            });

            if (!existingLog) {
                await db.collection('visitingLogs').insertOne({
                    agentName: agentName,
                    propertyId: new ObjectId(propertyid),
                    status: "Active",
                    entryTime: new Date(),
                    propertyName: property.name
                });
                return res.status(200).json({ message: "Welcome! Visit Started." });
            }
            return res.status(200).json({ message: "Agent in zone" });

        } else {
            const activeLog = await db.collection("visitingLogs").findOne({
                agentName: agentName,
```

Developed a "Status-Check" to ensure an agent isn't logged multiple times for the same visit.

Securing Sensitive Admin Data :

Problem: The Admin Dashboard handles property coordinates and agent visiting logs. Without authentication, anyone with the API endpoint could modify geofences or view private tracking data.

Solution: Implemented **JWT (JSON Web Token) Authentication**.

```
// ========================= verifying JWT ====================
const authenticateJWT = (req, res, next)=>{
    const token = req.cookies.token;
    if(!token){
        return res.status(401).json({message:"No token Found"})
    }
    jwt.verify(token, process.env.JWT_KEY,(err, decoded)=>{
        if(err){
            return res.status(401).json({message:"Unauthorized access"})
        }
        req.user = decoded;
        next();
    })
}
```

- **Stateless Security:** Created a secure login flow where the server issues a signed token upon successful admin authentication.
- **Protected Routes:** Developed custom **Backend Middleware** to verify the token's validity before allowing access to `POST`, `PATCH`, or `DELETE` end points.

Protecting Admin Credentials from XSS Attacks:

Problem: Initially, the JWT was stored in LocalStorage. However, this left the application vulnerable to Cross-Site Scripting (XSS), as any JavaScript running on the page could read the token and hijack the Admin session.

Solution: Secure Cookie-Based Authentication

```
> ////////////////////////////// Admin Login \\\\
```